

This **make-up** assignment is due **August 29, 2025**. There will be no extensions for this assignment under any circumstances. You may work in teams of up to four people. If you work as a team, choose a leader to sign up your team on the homework 4 groups page of Bruin Learn. Please submit one html or pdf file. Upload your solutions to Bruin Learn. Show all your work. You may use the modules `math`, `numpy`, `pandas`, `scipy`, and `matplotlib`.

1. The gradient descent algorithm shown below finds a local minimum of the function f given an initial guess x_0 . The value h is used for the numerical derivative approximation g . The learning rate η controls the step size taken in each iteration. A value of η too small can lead to slow convergence, while a value of η too large can cause divergence. The algorithm terminates when either the absolute difference between successive iterations is smaller than a tolerance α (indicating convergence), larger than a threshold β (indicating divergence), or when a maximum number of iterations N is reached.

```

Input:  $f, x_0, h, \eta, \alpha, \beta, N$ 
1 def  $g(x)$ :
2   | return  $\frac{f(x+h)-f(x-h)}{2h}$ ;
3 for  $i \leftarrow 1$  to  $N$  do
4   |  $x \leftarrow x_0 - \eta g(x_0)$ ;
5   | if  $|x - x_0| < \alpha$  then
6   |   | return  $x$ ;
7   | else if  $|x - x_0| > \beta$  then
8   |   | return  $NaN$ ;
9   | else
10  |   |  $x_0 \leftarrow x$ ;
11  | end
12 end
13 return  $NaN$ ;

```

- (a) Write a function that implements this algorithm in Python. Set respective default arguments for h, η, α, β , and N of $10^{-2}, 10^{-3}, 10^{-6}, 10^6$, and 100000.

For parts (b), (c), and (d), use $f(x) = 3x^4 - 16x^3 + 6x^2 + 72x + 5$. Make sure you interpret your results. Simply printing out numbers will not receive full credit.

- (b) Pick a few values of x_0 and use your function from (a) to find the local minima of f .
- (c) Repeat (b) with $-f$ instead of f to find the local maxima of f .
- (d) Verify your work by using `plot` and `scatter` in `matplotlib.pyplot` to plot f and your local extrema, respectively.

2. Use partition pairs of the form $P = (1, b^{1/n}, b^{2/n}, \dots, b)$ and $T = (1, b^{1/n}, b^{2/n}, \dots, b^{(n-1)/n})$ to analytically compute

$$\int_1^b \frac{dx}{x^p}$$

for $b, p > 1$.

3. You have graduated from the MFE program and landed yourself a job as a quantitative trading analyst in a pod shop! Through careful analysis, you determine that the best way to model the market impact of trades is via a function proportional to $x^{3/2}$, where x represents the size of the transaction as a percent. However, your shop uses quadratic programming for portfolio construction. Hence, to integrate a market impact model into the quadratic program, the model must be a quadratic polynomial. As a result, your goal is to approximate $x^{3/2}$ using a quadratic polynomial. Your pod shop does not make any trades larger than 1% of its portfolio as a matter of policy. Therefore, if $\hat{f}$ is your approximation, you are interested in reducing the error

$$\int_0^1 \left[x^{3/2} - \hat{f}(x) \right]^2 dx.$$

You want to find the orthogonal projection of $x^{3/2}$ onto the subspace of quadratic polynomials. Since you are interested in reducing the square error, use the inner product

$$\langle f, g \rangle = \int_0^1 f(x)g(x) dx.$$

- (a) Orthogonalize the basis $(1, x, x^2)$.
- (b) Use your result from (a) to project $x^{3/2}$ onto the subspace of polynomials of order at most two.
- (c) To assess model efficacy, also use a second-degree Taylor polynomial centered at $x = 1/2$ to approximate $x^{3/2}$.
- (d) Graph $x^{3/2}$ as well as your results from (b) and (c) on one plot via `plot` in `matplotlib.pyplot`. Make sure to add a legend.
- (e) Compute the norm between $x^{3/2}$ and your results from (b) and (c).

4. Within the Black-Scholes framework, the continuous return over T periods of a non-dividend paying stock is $X_T \sim \mathcal{N}\left((r - \frac{\sigma^2}{2})T, T\sigma^2\right)$, where r is the continuous risk-free rate of return. If the initial price of the stock is S_0 , it follows that the price at time T is the random variable $S_T = S_0 e^{X_T}$. Let us suppose $S_0 = 90$, $T = 2$, $r = 4\%$, and $\sigma = 45\%$.

- (a) Generate 100,000 values of X_T using `scipy.stats.norm.rvs`. Use `np.random.seed` to set the random seed to a particular number for reproducibility. Then find the corresponding values of S_T using the formula above. Use `plt.subplots` to create a figure with subplots of the histograms of X_T and S_T . For both histograms, set `bins = 20` and `density = True`. Place a title above each subplot.
- (b) We can use your results in part (a) to price a European call option for this stock. This option gives the owner the right, but not the obligation, to convert to equity for a price of K at maturity. The fair value of this option—given our assumptions—is

$$C = e^{-rT} E[\max(S_T - K, 0)].$$

Compute the payoff $\max(S_T - K, 0)$ for each of the values of S_T from (a). Assume $K = 100$ in your calculations. Take the mean to estimate the expected payoff. Lastly, discount the expected payoff to the present value using e^{-rT} .

- (c) The Black-Scholes formula for the value of a European call option is

$$C = S_0 \Phi(d_1) - K e^{-rT} \Phi(d_2),$$

where

$$d_1 = \frac{\ln\left(\frac{S_0}{K}\right) + \left(r + \frac{\sigma^2}{2}\right)T}{\sigma\sqrt{T}} \quad \text{and} \quad d_2 = d_1 - \sigma\sqrt{T}.$$

Use the formula to verify your result in (b). Because of the stochastic nature of the calculation in (b), your result in (c) may not be exactly the same.

- (d) Generate n values of X_T for each value of n in `np.logspace(2, 20, 1000, base = 2)`. Then for each n , repeat (b) to compute a corresponding estimate of the call price. Use `plot` in `matplotlib.pyplot` to plot your result as a function of n . On the same plot, include a red dashed line to display your answer in (c). Make sure to label your axes, and add a legend. You may want to reset the random seed so that you do not need to do the problems (a) and (d) sequentially.

5. (*Bonus Problem*) Within fixed income, investors essentially use first- and second-order Taylor polynomials to approximate the price of a bond given hypothetical changes in interest rates. Consider a bond with an annualized coupon rate paid semiannually of 4%, a par value of 100, and one year until maturity. If the continuously compounded discount rate is r , the price of the bond as a function of r is

$$P(r) = 2e^{-r/2} + 102e^{-r}.$$

- (a) Find the first- and second-order Taylor polynomials centered at $r = 6\%$ for $P(r)$. Use the Taylor polynomials to approximate $P(7\%) - P(6\%)$. Also, use the pricing formula directly to compute the value of the difference.
- (b) Compute the Lagrange error bounds for your two Taylor polynomials. Suppose that $5\% \leq r \leq 7\%$ when you perform your optimization.
- (c) Create two subplots. On the first subplot, graph $P(r)$ and the first-order Taylor polynomial you found in (a). Use `plt.fill_between` to depict the error bound for the first-order Taylor series you found in (b); use the `alpha` parameter to adjust the opacity of the error bound. For the second subplot, use the second-order Taylor polynomial and corresponding Lagrange error bound, and repeat what you did for the first subplot. In both subplots, make sure you add a legend to clarify the meanings of each curve.